

DIP HW5_1

313512009 施丞蔚

1. Please use a gradient computed in RGB color vector space to detect the edge for the image, 'lenna-RGB.tif' as Fig. 6.44(b) in pp.453. Please describe your method, procedures, final gradient image and print out the source code? (3X10=40)

```
import numpy as np
import cv2
from matplotlib import pyplot as plt
from scipy import ndimage # 如果沒有這個套件, 請先: pip3 install scipy

def rgb_vector_gradient_edge_detection(img_path,
save_path="lenna_RGB_gradient.png"):
    # 1. 用 OpenCV 讀取彩色影像 (BGR)
    img_bgr = cv2.imread(img_path, cv2.IMREAD_COLOR)
    if img_bgr is None:
        raise FileNotFoundError(f"找不到影像檔案: {img_path}")

    # 轉成 RGB, 並正規化到 [0,1] (float64)
    img = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
    img = img.astype(np.float64) / 255.0

    # 檢查影像形狀
    if img.ndim != 3 or img.shape[2] != 3:
        raise ValueError("輸入影像必須是 RGB 彩色影像 (H x W x 3)")

    # 2. 定義 Sobel 濾波器 (水平方向 Sx 與垂直方向 Sy)
    Sx = np.array([[ -1,  0,  1],
                    [-2,  0,  2],
                    [-1,  0,  1]], dtype=float)

    Sy = np.array([[ 1,  2,  1],
                    [ 0,  0,  0],
                    [-1, -2, -1]], dtype=float)

    # 預先建立 Gx, Gy 陣列, 用來存放每個通道的梯度
    Gx = np.zeros_like(img, dtype=float) # H x W x 3
    Gy = np.zeros_like(img, dtype=float)

    # 3. 對 R、G、B 三個通道分別做卷積, 得到每一通道的 x 與 y 項梯度
```

```

for c in range(3): # c = 0(R),1(G),2(B)
    Gx[:, :, c] = ndimage.convolve(img[:, :, c], Sx, mode='reflect')
    Gy[:, :, c] = ndimage.convolve(img[:, :, c], Sy, mode='reflect')

# 4. 在 RGB 向量空間計算梯度大小
# ||Gx||^2 = (dR/dx)^2 + (dG/dx)^2 + (dB/dx)^2
# ||Gy||^2 = (dR/dy)^2 + (dG/dy)^2 + (dB/dy)^2
Gx_norm2 = np.sum(Gx ** 2, axis=2) # H x W
Gy_norm2 = np.sum(Gy ** 2, axis=2) # H x W

# ||G|| = sqrt( ||Gx||^2 + ||Gy||^2 )
grad_mag = np.sqrt(Gx_norm2 + Gy_norm2)

# 5. 正規化到 [0,1]
grad_mag = grad_mag / (grad_mag.max() + 1e-12)

# 6. 顯示結果
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(img)
plt.title("Original RGB image")
plt.axis("off")

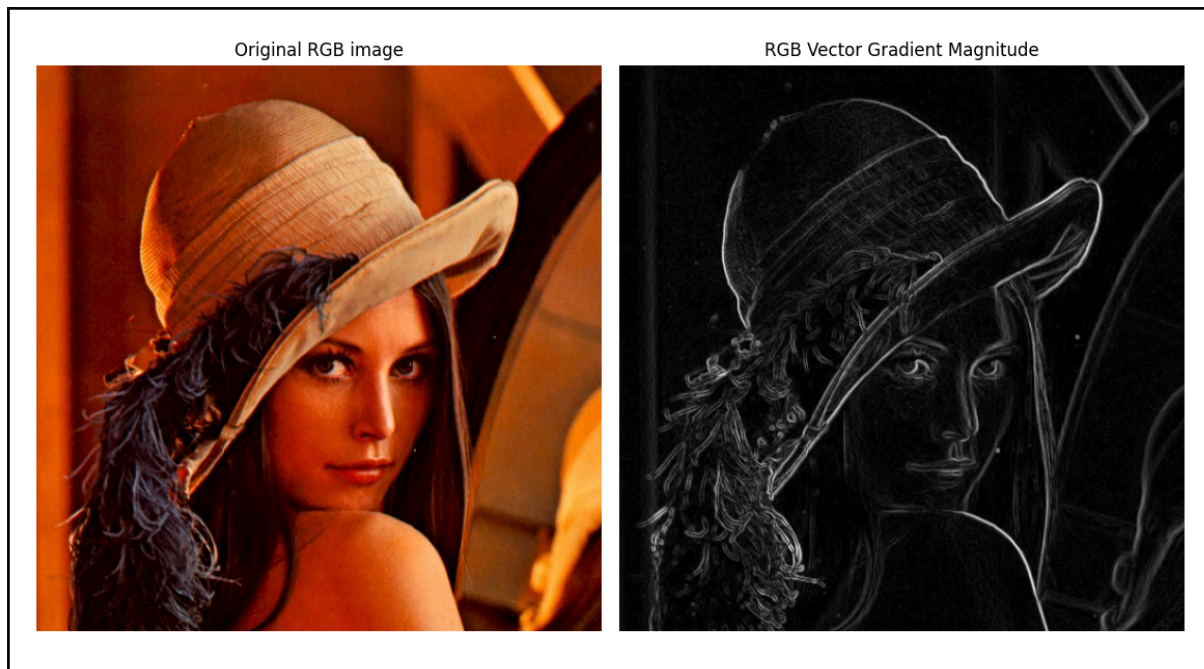
plt.subplot(1, 2, 2)
plt.imshow(grad_mag, cmap="gray")
plt.title("RGB Vector Gradient Magnitude")
plt.axis("off")

plt.tight_layout()
plt.show()

# 7. 儲存結果為 8-bit 灰階影像
grad_mag_uint8 = (grad_mag * 255).astype(np.uint8)
cv2.imwrite(save_path, grad_mag_uint8)
print(f"已將彩色梯度邊緣影像儲存為：{save_path}")

if __name__ == "__main__":
    # lenna-RGB.tif 放在同一個資料夾就直接這樣寫
    rgb_vector_gradient_edge_detection("lenna-RGB.tif")

```



方法概念

我把每一個畫素視為 RGB 向量 $[R, G, B]$, 在三個色彩通道上分別做 Sobel 微分, 得到在 x 、 y 方向的向量梯度, 然後利用三個通道梯度的平方和來計算一個彩色梯度大小作為邊緣強度。

主要步驟

1. 使用 OpenCV 讀入 `lenna-RGB.tif`, 從 BGR 轉成 RGB, 再把像素值正規化到 $[0,1][0,1][0,1]$ 的浮點數。
2. 定義 Sobel 水平與垂直遮罩 S_x 、 S_y 。
3. 對 R、G、B 三個通道各自與 S_x 、 S_y 做 2D 卷積, 得到每一通道在 x 、 y 方向上的梯度 $G_x(R, G, B)$ 、 $G_y(R, G, B)$ 。
4. 對每個像素, 先把三個通道的梯度平方後相加, 得到 $\|G_x\|^2$ 、 $\|G_y\|^2$, 再取平方根 $(\|G_x\|^2 + \|G_y\|^2)^{1/2}$ 當成 RGB 向量梯度的大小。
5. 將梯度大小影像用最大值做正規化到 $[0,1]$, 一方面用 `matplotlib` 顯示, 一方面乘上 255 轉成 8-bit 灰階, 存成 `lenna_RGB_gradient.png`。

最終梯度影像的特性

最後得到的是一張灰階邊緣圖。Lena 影像中輪廓明顯的地方，例如臉部輪廓、帽子邊緣、頭髮與背景交界處等，因為 RGB 向量變化大，所以在梯度影像中呈現較亮的線條；平坦且顏色變化小的區域則接近黑色。這個結果同時反映亮度與色彩變化，因此比單純灰階梯度更能捕捉顏色差異產生的邊界。

2. Repeat (1) steps in the image ‘Visual resolution.gif’? (3X10=40)

```

import numpy as np
import cv2
from matplotlib import pyplot as plt
import matplotlib.image as mpimg # 新增：用來讀 GIF

def rgb_vector_gradient_edge_detection(img_path,
save_path="Visual_resolution_gradient.png"):
    # 1. 讀取 GIF 影像, 用 matplotlib 支援 GIF
    img = mpimg.imread(img_path) # 可能是 uint8 或 float、RGB 或 RGBA

    # 如果有 alpha 通道 (RGBA), 只取前 3 個通道
    if img.ndim == 3 and img.shape[2] == 4:
        img = img[:, :, :3]

    if img.ndim != 3 or img.shape[2] != 3:
        raise ValueError("輸入影像必須是 RGB 彩色影像 (H x W x 3)")

    # 轉成 float64, 範圍 [0,1]
    if img.dtype == np.uint8:
        img = img.astype(np.float64) / 255.0
    else:
        img = img.astype(np.float64)

    # 2. 定義 Sobel 濾波器 (水平方向 Sx 與垂直方向 Sy)
    Sx = np.array([[ -1,  0,  1],
                    [ -2,  0,  2],
                    [ -1,  0,  1]], dtype=float)

    Sy = np.array([[ 1,  2,  1],
                    [ 0,  0,  0],
                    [-1, -2, -1]], dtype=float)

    # 建立 Gx, Gy 陣列
    Gx = np.zeros_like(img, dtype=float)
    Gy = np.zeros_like(img, dtype=float)

    # 3. 對 R、G、B 三個通道分別做卷積
    # 使用 cv2.filter2D
    for c in range(3): # 0:R, 1:G, 2:B
        Gx[:, :, c] = cv2.filter2D(img[:, :, c], -1, Sx,
borderType=cv2.BORDER_REFLECT)

```

```

        Gy[:, :, c] = cv2.filter2D(img[:, :, c], -1, Sy,
borderType=cv2.BORDER_REFLECT)

# 4. 在 RGB 向量空間計算梯度大小
Gx_norm2 = np.sum(Gx ** 2, axis=2)    # H x W
Gy_norm2 = np.sum(Gy ** 2, axis=2)    # H x W
grad_mag = np.sqrt(Gx_norm2 + Gy_norm2)

# 5. 正規化到 [0,1]
grad_mag = grad_mag / (grad_mag.max() + 1e-12)

# 6. 顯示結果
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(img)
plt.title("Original Visual resolution image")
plt.axis("off")

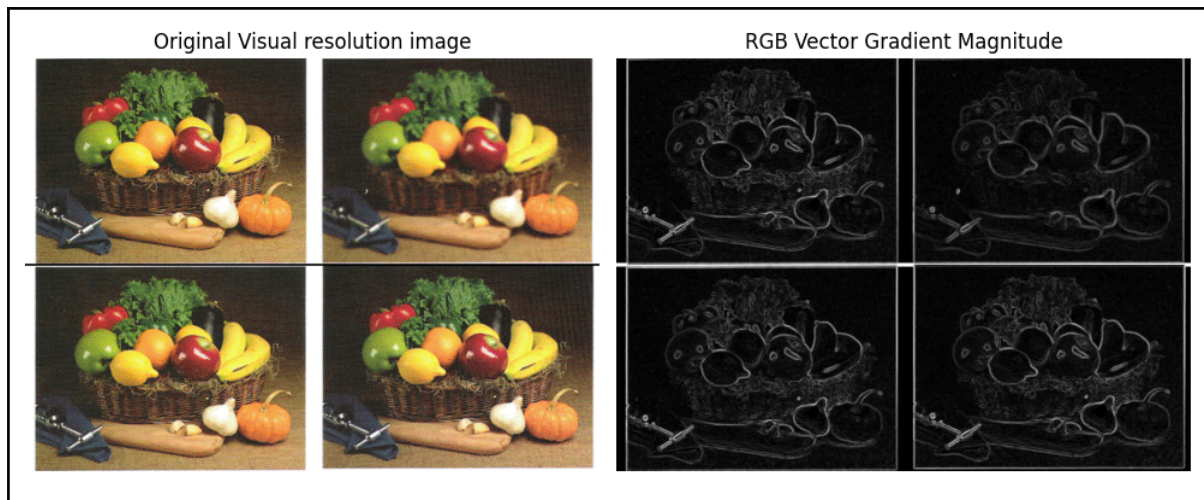
plt.subplot(1, 2, 2)
plt.imshow(grad_mag, cmap="gray")
plt.title("RGB Vector Gradient Magnitude")
plt.axis("off")

plt.tight_layout()
plt.show()

# 7. 儲存結果
grad_mag_uint8 = (grad_mag * 255).astype(np.uint8)
cv2.imwrite(save_path, grad_mag_uint8)
print(f"已將彩色梯度邊緣影像儲存為：{save_path}")

if __name__ == "__main__":
    # 跟 q2.py 在同一個資料夾的情況
    rgb_vector_gradient_edge_detection("Visual resolution.gif")

```



方法概念

第二題沿用完全相同的 RGB 向量梯度計算方式，只是將輸入影像改成 [Visual resolution.gif](#)。這張圖包含四張水果籃小圖，我把整張 GIF 當成一張 RGB 影像來計算 Sobel 向量梯度並產生邊緣圖。

主要步驟

1. 使用 `matplotlib.image.imread` 讀取 [Visual resolution.gif](#)，若為 RGBA 影像則丟掉 alpha 通道，保留前三個 RGB 通道，並正規化到 [0,1]。
2. 一樣定義 Sobel 遮罩 S_x 、 S_y 。
3. 對 R、G、B 三個通道分別用 `cv2.filter2D` 與 S_x 、 S_y 做卷積，計算出每一通道在 x、y 方向的梯度。
4. 對每個像素計算 $\|G_x\|^2$ 、 $\|G_y\|^2$ 並取 $(\|G_x\|^2 + \|G_y\|^2)^{1/2}$ 得到彩色梯度大小。

5. 將梯度大小影像以最大值正規化到 $[0,1]$ ，用 `matplotlib` 顯示原圖與梯度圖，並將結果乘 255 後存成 8-bit 灰階影像

`Visual_resolution_gradient.png`。

最終梯度影像的特性

輸出的梯度影像中可以清楚看到四個子圖各自的蔬果輪廓、籃子邊緣、桌面與背景交界。這些地方因亮度或色彩改變劇烈而產生較大的 RGB 向量梯度，因此在結果中呈現明亮的邊緣線；相對平坦的桌面與背景區域則較暗。四個子圖的解析度或品質差異，會反映在邊緣清晰度與細節強度上，方便比較不同視覺解析度對邊緣偵測結果的影響。

3. Please comment and compare your two designs? (20)

Design 1 (Lenna)

- 影像內容是單一場景、解析度較高、細節很多(帽子、頭髮、籃子紋理、背景等)。
- 在 RGB 向量空間計算 Sobel 梯度後, 可以看到人物輪廓、帽子邊緣、頭髮與背景交界、眼睛與嘴巴等細節都有亮的梯度反應, 邊緣細而清楚。
- 因為圖像本身訊噪比好、顏色與亮度變化都蠻平順, 所以梯度圖中雜訊不多, 主要是物體邊界被強調出來, 非常適合作為展示「彩色向量梯度」的例子。

Design 2 (Visual resolution)

- 影像是一張包含四個水果籃子的小圖, 通常代表不同「視覺解析度」或不同品質版本;我的梯度運算是直接對這張大圖整體做 RGB 向量梯度。
- 在梯度結果中, 四個子圖的邊緣都被偵測出來, 但可以觀察到:解析度較高、細節較豐富的那一塊, 籃子編織紋理、葉片紋路、蔬果輪廓都比較細緻、亮度變化也比較銳利;解析度較低或較模糊的那幾塊, 梯度主要只剩下比較粗的物體輪廓, 細小紋理幾乎消失, 邊緣也變得較厚、較模糊。
- 因為原圖有壓縮與畫質差異, 有些區塊在顏色量化或雜訊較明顯的地方, 梯度圖會出現比較雜亂的小亮點, 顯示出 RGB 梯度對色彩變化也很敏感。

比較

第一個設計(lenna-RGB.tif)使用 RGB 向量梯度搭配 Sobel 濾波, 在三個色彩通道上分別微分再合成梯度大小。結果中可以清楚看到人物輪廓、帽子、頭髮與背景交界等邊緣, 線條細緻、雜訊少, 主要強調物體幾何輪廓與部分紋理, 是一個較乾淨的彩色邊緣圖。

第二個設計(Visual resolution.gif)採用完全相同的 RGB 向量梯度方法, 但輸入是一張包含四個子圖、解析度不同的水果影像。梯度結果中四塊區域的籃子與水果輪廓都被偵測出來, 高解析度那一塊邊緣細節較多、線條銳利, 低解析度區塊則只剩較粗的邊緣且較模糊。